19.3 出站消息适配

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~560k tokens，输出 ~50k tokens（本章合计）

AI Agent 生成的回复是标准 Markdown 格式——但不同通道对 Markdown 的支持程度大相径庭。Telegram 支持粗体、斜体、代码块和链接；WhatsApp 只支持有限的格式标记；Discord 支持丰富的 Markdown 但有 2000 字符限制。出站消息适配层的任务，就是将统一的 Markdown 回复转换为每个通道能够正确渲染的格式。

19.3.1 Markdown 格式化与通道特异性

中间表示（IR）架构

OpenClaw 没有为每个通道单独编写 Markdown 解析器，而是采用了**中间表示（Intermediate Representation, IR）**架构。这是编译器设计中的经典模式：

```
Markdown 源文本
    │
    ▼ 解析（markdown-it）
MarkdownIR（中间表示）
    │
    ├── 渲染为 Telegram 格式
    ├── 渲染为 Discord 格式
    ├── 渲染为 WhatsApp 格式
    └── 渲染为纯文本格式
```

> **衍生解释：中间表示（Intermediate Representation）**
>
> 在编译器设计中，中间表示是源语言和目标语言之间的桥梁数据结构。经典编译器（如 LLVM）将源代码先编译为 IR，再从 IR 生成不同目标平台的机器码。这样只需要 N 个前端 + M 个后端，而不是 N×M 个翻译器。OpenClaw 的 Markdown IR 同理：一个解析器 + K 个通道渲染器，而不是 K 个独立的解析 + 渲染流程。

`MarkdownIR` 的数据结构定义在 `src/markdown/ir.ts`：

```
// src/markdown/ir.ts
export type MarkdownStyle = "bold" | "italic" | "strikethrough" | "code" | "code_block"

export type MarkdownStyleSpan = {
  start: number;        // 在纯文本中的起始位置
  end: number;          // 在纯文本中的结束位置
  style: MarkdownStyle; // 样式类型
};

export type MarkdownLinkSpan = {
  start: number;
  end: number;
  href: string;         // 链接目标 URL
};

export type MarkdownIR = {
  text: string;                     // 纯文本内容（无格式标记）
  styles: MarkdownStyleSpan[];      // 样式区间列表
  links: MarkdownLinkSpan[];        // 链接区间列表
};
```

IR 的核心思想是将**内容**和**格式**分离：`text` 字段存储纯文本，`styles` 和 `links` 数组以区间（span）的形式记录哪些文本范围应用了什么样式。这种"标注式"表示比嵌套 AST 更适合跨平台渲染——每个渲染器只需要知道如何将样式区间转换为目标格式的标记。

解析器：markdown-it

IR 的构建使用 `markdown-it` 库——一个高性能的 Markdown 解析器：

```
// src/markdown/ir.ts（简化）
function createMarkdownIt(options: MarkdownParseOptions): MarkdownIt {
  const md = new MarkdownIt({
    html: false,              // 禁用 HTML 标签（安全考量）
    linkify: options.linkify ?? true,  // 自动识别 URL
    breaks: false,            // 不将换行转为 <br>
    typographer: false,       // 不进行排版美化
  });
  md.enable("strikethrough");

  // 表格支持按需启用
  if (options.tableMode && options.tableMode !== "off") {
    md.enable("table");
  } else {
    md.disable("table");
  }

  return md;
}
```

渲染器：样式标记映射

渲染器的核心是 `renderMarkdownWithMarkers`（位于 `src/markdown/render.ts`），它将 IR 转换为带有通道特定标记的文本：

```
// src/markdown/render.ts（简化）
export type RenderStyleMarker = {
  open: string;      // 开始标记
  close: string;     // 结束标记
};

export type RenderStyleMap = Partial<Record<MarkdownStyle, RenderStyleMarker>>;

export function renderMarkdownWithMarkers(ir: MarkdownIR, options: RenderOptions): stri
  // 1. 收集所有样式区间的边界点
  // 2. 在每个边界点插入对应的开/闭标记
  // 3. 处理嵌套和重叠的样式区间
  // ...
}
```

不同通道的样式标记对比：

样式	Markdown 原始	Telegram	WhatsApp	Discord	纯文本
粗体	`**text**`	`**text**`	`*text*`	`**text**`	text
斜体	`*text*`	`_text_`	`_text_`	`*text*`	text
删除线	`~~text~~`	`~~text~~`	`~text~`	`~~text~~`	text
行内代码	`` `code` ``	`` `code` ``	`` `code` ``	`` `code` ``	code
代码块	```` ```code``` ````	```` ```code``` ````	```` ```code``` ````	```` ```code``` ````	code

表格处理

Markdown 表格是最难跨平台渲染的元素之一。OpenClaw 支持三种表格渲染模式：

```
// src/config/types.base.ts
export type MarkdownTableMode = "off" | "bullets" | "code";
```

模式	效果	适用通道
`off`	不处理，保留原始 Markdown 表格	Telegram、Discord（原生支持）
`bullets`	转换为列表格式	WhatsApp、Signal（不支持表格）
`code`	转换为等宽代码块	需要精确对齐的场景

表格转换由 `convertMarkdownTables` 函数处理：

```
// src/markdown/tables.ts
export function convertMarkdownTables(markdown: string, mode: MarkdownTableMode): strin
  if (!markdown || mode === "off") return markdown;

  const { ir, hasTables } = markdownToIRWithMeta(markdown, {
    tableMode: mode,
    // ...
  });
  if (!hasTables) return markdown;    // 快速路径：无表格则不处理

  return renderMarkdownWithMarkers(ir, {
    styleMarkers: MARKDOWN_STYLE_MARKERS,
    escapeText: (text) => text,
    // ...
  });
}
```

嵌套样式的正确处理

渲染器必须正确处理样式的嵌套和重叠。例如 `**bold _bold-italic_ bold**` 包含嵌套的粗体和斜体。 `renderMarkdownWithMarkers` 使用了**栈式状态管理**：

```
// src/markdown/render.ts（简化）
// 统一的栈，追踪关闭标记和结束位置
const stack: { close: string; end: number }[] = [];

for (const pos of boundaryPoints) {
  // 在当前位置关闭所有已结束的样式（LIFO 顺序）
  while (stack.length && stack[stack.length - 1]?.end === pos) {
    out += stack.pop().close;
  }

  // 在当前位置打开新的样式（按结束位置降序排列，外层先打开）
  for (const item of openingItems) {
    out += item.open;
    stack.push({ close: item.close, end: item.end });
  }

  // 输出当前位置到下一个边界之间的文本
  out += escapeText(text.slice(pos, nextPos));
}
```

这个栈确保了关闭标记以 LIFO（后进先出）顺序插入——外层样式最先打开、最后关闭，内层样式最后打开、最先关闭。这与 HTML/XML 的嵌套规则一致。

19.3.2 回复前缀

动态前缀注入

OpenClaw 支持在 AI 回复的开头注入**回复前缀（Response Prefix）**——一段可配置的模板文本，通常包含 Agent 名称、使用的模型等信息。这一功能通过 `src/channels/reply-prefix.ts` 实现：

```ts
// src/channels/reply-prefix.ts（简化）
export type ResponsePrefixContext = {
  identityName: string;          // Agent 身份名称
  provider?: string;             // 模型提供者（如 "anthropic"）
  model?: string;                // 模型短名（如 "opus-4"）
  modelFull?: string;            // 完整模型名（如 "anthropic/claude-opus-4"）
  thinkingLevel?: string;        // 思考等级（如 "off", "medium"）
};

export function createReplyPrefixContext(params: {
  cfg: OpenClawConfig;
  agentId: string;
  channel?: string;
  accountId?: string;
}): ReplyPrefixContextBundle {
  const prefixContext: ResponsePrefixContext = {
    identityName: resolveIdentityName(cfg, agentId),
  };

  const onModelSelected = (ctx: ModelSelectionContext) => {
    // 直接修改对象（而非重新赋值），确保闭包能看到更新
    prefixContext.provider = ctx.provider;
    prefixContext.model = extractShortModelName(ctx.model);
    prefixContext.modelFull = `${ctx.provider}/${ctx.model}`;
    prefixContext.thinkingLevel = ctx.thinkLevel ?? "off";
  };

  return {
    prefixContext,
    responsePrefix: resolveEffectiveMessagesConfig(cfg, agentId, {
      channel: params.channel,
      accountId: params.accountId,
    }).responsePrefix,
    responsePrefixContextProvider: () => prefixContext,
    onModelSelected,
  };
}
```

延迟绑定模式

注意 `onModelSelected` 回调的设计——它在模型选定后才填充 `provider` 、 `model` 等字段。这是因为：

1. 回复前缀的模板在**调度开始前**就需要准备好
2. 但使用的模型在**路由和故障转移之后**才确定（参见第 8 章）
3. 因此采用"先创建上下文对象，后填充模型信息"的**延迟绑定**模式

```
dispatchReply 开始
    │
    ▼
createReplyPrefixContext()          ← 创建空的前缀上下文
    │ prefixContext = { identityName: "Aria" }
    ▼
模型选择 & 故障转移
    │
    ▼
onModelSelected(ctx)                ← 回调填充模型信息
    │ prefixContext.model = "opus-4"
    ▼
渲染回复前缀模板
    │ "Aria (opus-4): "
    ▼
拼接到回复文本前面
```

这里有一个微妙但重要的实现细节： `onModelSelected` 通过**直接修改对象属性**（而不是重新赋值引用）来更新上下文。这确保了所有持有该对象引用的闭包都能看到更新——如果使用 `prefixContext = { ...newData }` 的方式，旧的闭包仍然引用旧对象。

19.3.3 会话标签

会话标签的作用

会话标签（Conversation Label）是一个人类可读的字符串，用于标识当前对话的"身份"。它出现在系统提示词中，帮助 AI Agent 理解"我在跟谁说话"。

```ts
// src/channels/conversation-label.ts（简化）
export function resolveConversationLabel(ctx: MsgContext): string | undefined {
  // 1. 显式标签优先
  const explicit = ctx.ConversationLabel?.trim();
  if (explicit) return explicit;

  // 2. 线程标签次之
  const threadLabel = ctx.ThreadLabel?.trim();
  if (threadLabel) return threadLabel;

  // 3. 私聊：使用发送者名称
  const chatType = normalizeChatType(ctx.ChatType);
  if (chatType === "direct") {
    return ctx.SenderName?.trim() || ctx.From?.trim() || undefined;
  }

  // 4. 群聊：使用群名 + 可选的 ID 后缀
  const base = ctx.GroupChannel?.trim()
    || ctx.GroupSubject?.trim()
    || ctx.GroupSpace?.trim()
    || ctx.From?.trim()
    || "";
  if (!base) return undefined;

  // 对于纯数字 ID 或 WhatsApp 群组 JID，附加 id 后缀
  const id = extractConversationId(ctx.From);
  if (id && shouldAppendId(id) && base !== id && !base.includes(id)) {
    return `${base} id:${id}`;
  }

  return base;
}
```

标签解析的优先级

解析按照明确性递减的顺序：

优先级	来源	示例
1	`ConversationLabel` （显式设置）	`"产品讨论群"`
2	`ThreadLabel` （线程标签）	`"#general / bug-report"`

优先级	来源	示例
3	私聊： SenderName 或 From	"张三"
4	群聊： GroupChannel / GroupSubject / GroupSpace	"产品团队"

ID 后缀的智能附加

对于群聊标签，函数会智能判断是否需要附加 ID 后缀：

```
// src/channels/conversation-label.ts
function shouldAppendId(id: string): boolean {
  if (/^[0-9]+$/.test(id)) return true;       // 纯数字 ID（如 Telegram 群组）
  if (id.includes("@g.us")) return true;       // WhatsApp 群组 JID
  return false;
}
```

但不是所有情况都附加。以下场景会跳过：

```
// 已包含 ID → 不重复
if (base.includes(id)) return base;

// 已有 "id:" 标注 → 不重复
if (base.toLowerCase().includes(" id:")) return base;

// 以 # 或 @ 开头（如 Discord 频道名）→ 本身已经是唯一标识
if (base.startsWith("#") || base.startsWith("@")) return base;
```

这些规则确保标签既有足够的信息量（AI 能区分不同对话），又不会过度冗余。

标签在系统提示词中的使用

会话标签最终被注入到系统提示词中，帮助 AI Agent 理解对话上下文。例如：

```
你正在与"张三 (+8613812345678)"通过 WhatsApp 私聊。
```

或群聊场景：

```
你正在"产品团队 id:120363012345678901@g.us"的 WhatsApp 群组中。
当前发言者是"张三"。
```

这使得 AI Agent 在多对话并发场景下，能够区分不同的对话并维持正确的上下文。

本节小结

1. **Markdown IR 架构**采用编译器中间表示模式，将解析和渲染解耦——一个解析器 + K 个通道渲染器，避免 N×M 的组合爆炸。
2. **样式标记映射**使每个通道可以用自己的格式标记（如 WhatsApp 用 `*` 表示粗体而非 `**`）渲染相同的 IR。
3. **表格转换**支持三种模式（off/bullets/code），适应不同通道对表格的支持程度。
4. **回复前缀**使用延迟绑定模式——先创建上下文对象，模型选定后通过回调填充模型信息，确保闭包引用的一致性。
5. **会话标签**按四级优先级解析（显式 > 线程 > 私聊发送者 > 群聊名称），并智能附加 ID 后缀以确保唯一性。
6. 出站消息适配层是"最后一公里"的格式转换——确保 AI 生成的标准 Markdown 在每个通道上都能正确、美观地呈现。